

Infosys Springboard

Kasireddy Seshi Reddy

Milestone 3

Project Title: Interactive Analytics Dashboard for Global Income Distribution

Milestone Requirements

Global Income Inequality Portal Development

Documentation & Technical Guide

A comprehensive breakdown of features, technology, and development methodology for the interactive socioeconomic research dashboard.

Live Deployment: <https://global-income-distribution.vercel.app>

Credentials for accessing Admin Portal: Mail ID: bunnyseshireddy@gmail.com

Password: Sesh@23082004

GitHub Repository: <https://github.com/Kasireddy-Seshi-Reddy/global-income-distribution>

1. Overview & Objective

The **Global Income Inequality Portal** is a premium, research-grade, full-stack web application designed to analyze and visualize global income concentration and inclusive development momentum.

The core objective of this platform is to transform raw socioeconomic datasets into an interactive, visually stunning, and easily digestible experience for researchers, policy makers, and students. It acts as both a public-facing presentation tool and a secure, personalized portal for authenticated data analysts.

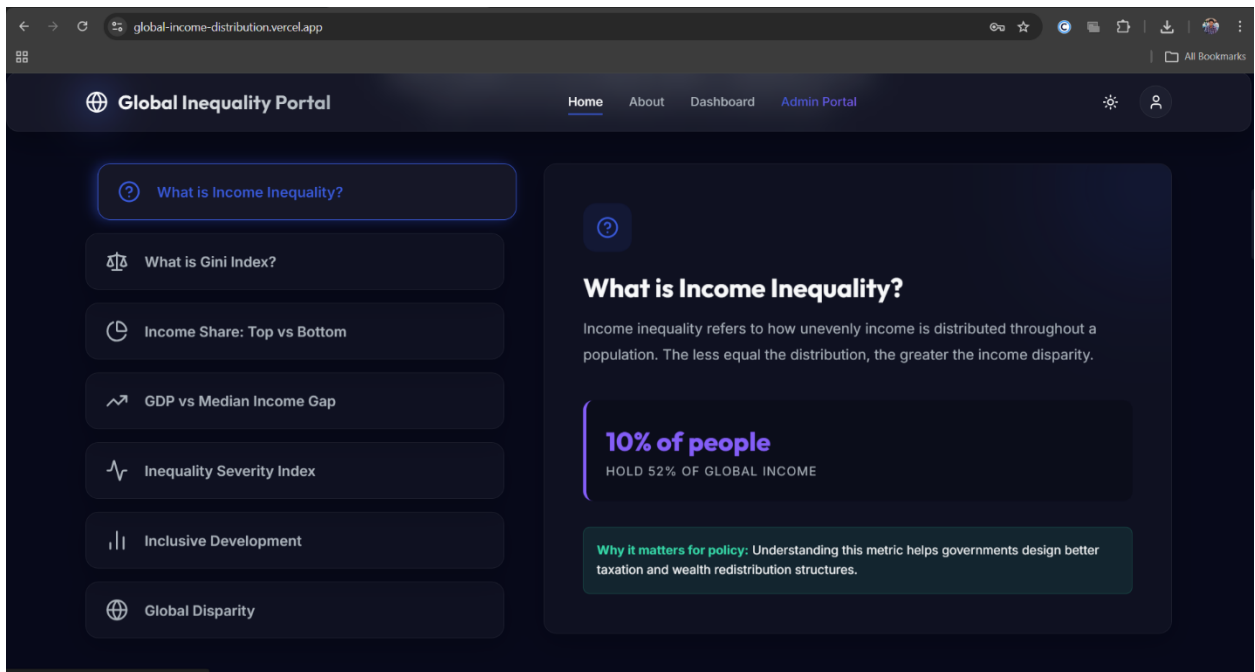
This platform was developed as part of the Infosys Springboard project framework.

2. What Users Experience (Features & Content)

The platform is designed with a modern **Glassmorphism** aesthetic, utilizing deep dark-mode themes, vibrant neon accents, and smooth micro-animations to create a highly engaging, premium feel.

Public Facing Areas

1. **Interactive Homepage:** A dynamic landing page that introduces the core problem of global wealth inequality with stunning entrance animations and responsive parallax effects.
2. **About Inequality:** Educational sections defining key terms (e.g., The Gini Coefficient, Wealth Concentration).
3. **Visualizations Explained:** A guide helping users interpret complex charts and data points.
4. **Contact the Analyst:** A secure feedback form allowing users to submit queries, which are routed directly to the database. Includes dynamic links to GitHub and LinkedIn.

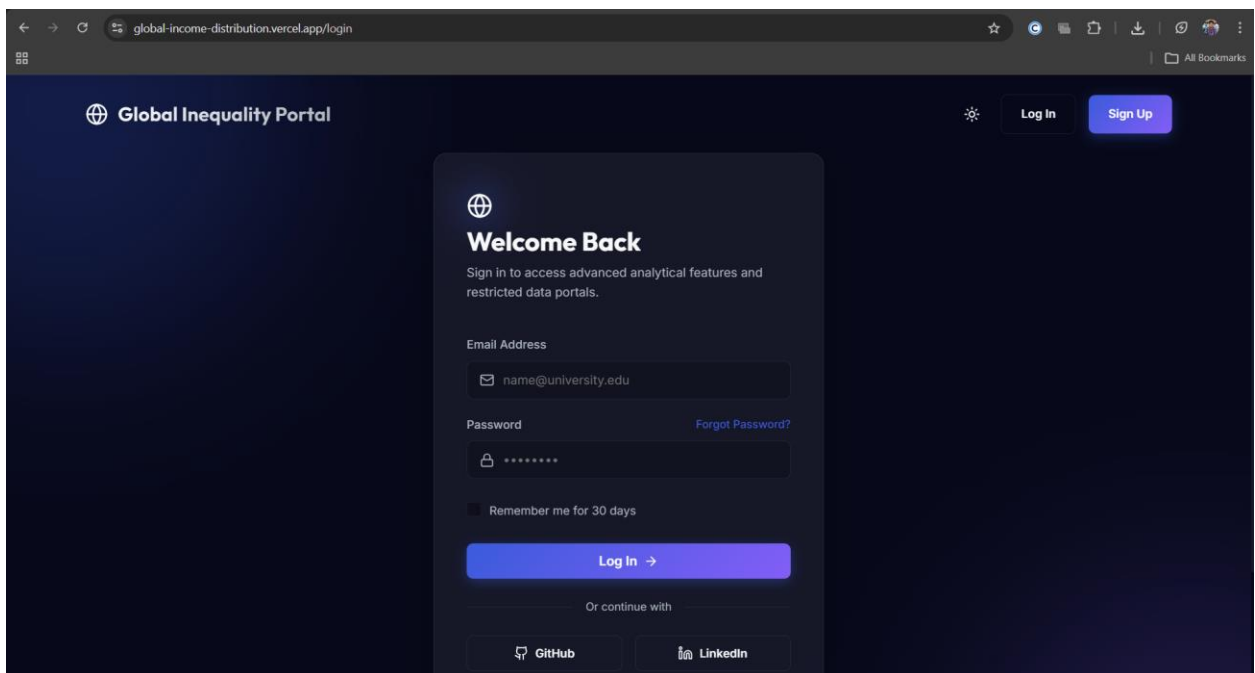


Authenticated Areas (Login Required)

1. **Secure Authentication:** Users can create accounts and log in using a custom Email/Password system backed by JWT token security.
2. **Interactive Power BI Dashboard (Seamless Embedding):** The centerpiece of the application. A massive, fully interactive data visualization canvas is seamlessly embedded directly into the React routing architecture.

User Interaction: Instead of downloading static charts, users can actively interact with the data directly inside the browser. By clicking on specific countries on the map, sliding timeline scales, or clicking specific income brackets, the entire dashboard dynamically filters and recalculates the surrounding charts in real-time, providing a frictionless, native analytical experience.

3. **Country Comparison Tool:** Analytical tools to pit different socioeconomic regions against one another.
4. **Data Export & Downloads:** Authenticated users can download raw spreadsheets or research reports.
5. **Session Tracking:** The application silently tracks user page views, login frequency, and session duration to generate usage heatmaps.

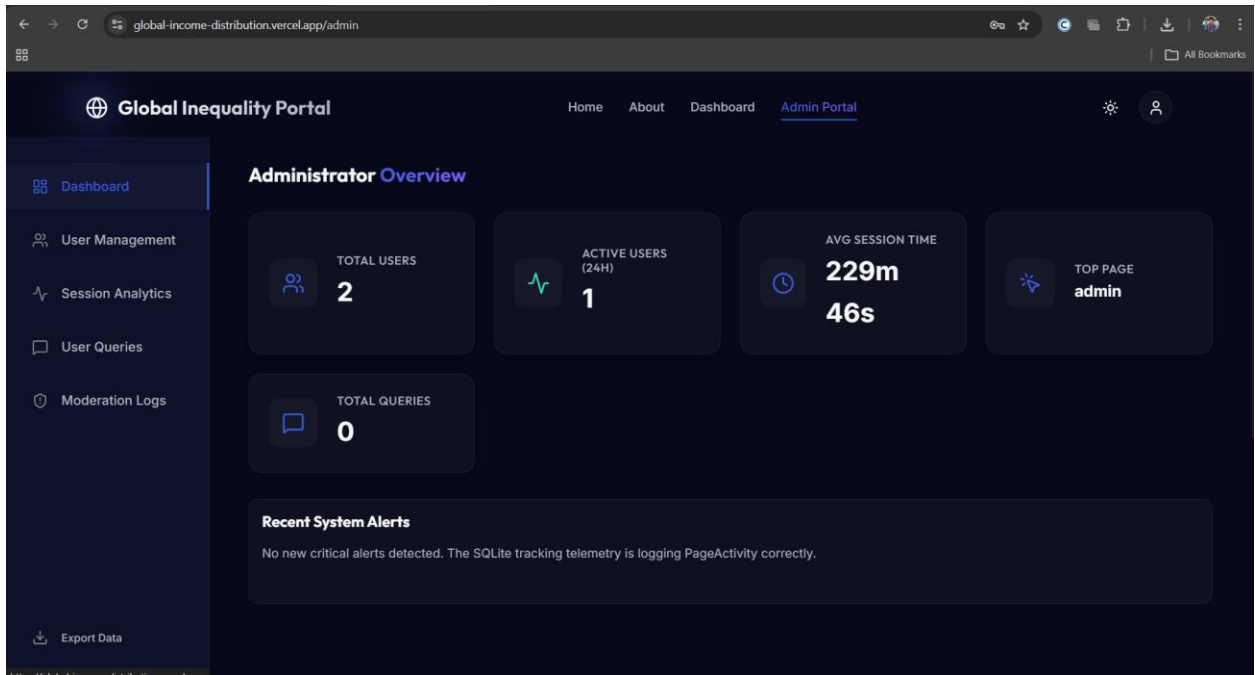


Admin Portal (Restricted Access)

Restricted exclusively to the project lead.

1. **KPI Dashboard:** Real-time metrics on total registered users, active sessions, and active queries.
2. **User Management:** An interactive data grid allowing the Admin to view all registered accounts, suspend users, Kick users offline, or permanently Ban them.
3. **Moderation Logs:** An audit trail of all disciplinary actions taken against accounts.
4. **Query Management:** A unified inbox to read, reply to, and resolve feedback submitted via the Contact form.

5. **CSV Exports:** One-click functionality to dump the live SQLite database into downloadable Excel sheets.



3. Technology Stack

To ensure maximum performance, security, and ease of deployment, a decoupled **Full-Stack JavaScript Architecture** was chosen.

Frontend (Client-Side)

- **React (Vite):** Powering the dynamic User Interface with lightning-fast hot reloading.
- **React Router:** Handles seamless Single Page Application (SPA) navigation without reloading the browser.
- **Vanilla CSS:** Custom Glassmorphism styling, CSS Variables (--primary-color), and Keyframe animations. Chosen over Tailwind to ensure pixel-perfect control over the premium dark theme.
- **Lucide React:** Scalable, lightweight SVG iconography used throughout the interface.
- **Context API:** Global state management for User Authentication and Theme Toggling (Dark/Light mode).

Backend (Server-Side)

- **Node.js & Express:** The robust, JavaScript-based server framework handling API routing and middleware.
- **SQLite3:** A highly efficient, file-based relational database. Selected because it requires zero external setup or cloud management, making local development and porting effortless.
- **JSON Web Tokens (JWT):** Stateless security tokens used to verify user identity on protected routes (like the Dashboard and Admin Portal).
- **Bcrypt.js:** Cryptographic hashing library used to securely salt and scramble user passwords before they are stored in the database.
- **CORS & Dotenv:** Middleware for handling cross-origin requests from the frontend and securely loading hidden environment variables.

Deployment & Hosting

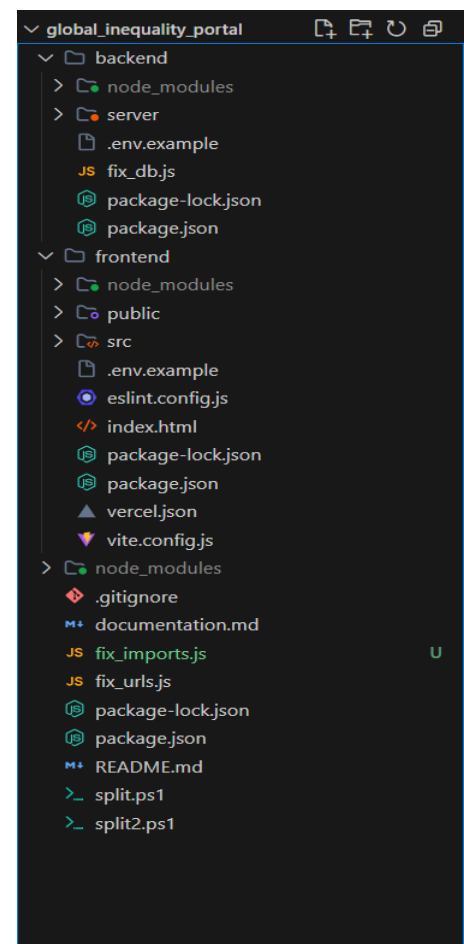
- **Monorepo Structure:** The codebase is split into /frontend and /backend directories, allowing them to be deployed independently.
- **Frontend Hosting (Vercel):** Optimized for serving React SPAs with continuous GitHub integration.
- **Backend Hosting (Render):** A cloud platform capable of running the Node.js Express server and maintaining the persistent SQLite database file.

4. Development Methodology & Implementation

The application was built systematically through rapid, phase-based continuous integration. Here is a compact overview of how the development tasks were executed:

Phase 1: UX/UI Design & Frontend Foundation

1. **Scaffolding:** Initialized the React application using Vite for speed.
2. **Design System:** Built index.css to define the global color variables, glass-panel mixins, and gradient text utilities.
3. **Component Architecture:** Broke the UI down into reusable parts: Navbar, Footer, AuthContainer, and HeroSection.
4. **Power BI Embed Integration:** Integrated the published Power BI report directly into the React Dashboard component using a responsive iframe.



5. **Routing:** Set up react-router-dom to map URLs (e.g., /login, /dashboard) to specific React components.

Phase 2: Database Design & Backend Scaffolding

1. **Express Setup:** Created a basic Node server running on port 5000.
2. **Schema Definition:** Designed precise SQLite tables to hold the data:
 - Users (Credentials and Roles)
 - UserSessions (Login/Logout timestamps)
 - PageActivity (Time spent per unique URL)
 - UserQueries (Contact form messages)
3. **API Routes:** Built the Express listening endpoints (e.g., POST /api/auth/login) to receive frontend data and read/write to the SQLite tables.

Phase 3: Security & Authentication Integration

1. **Password Protection:** Implemented bcrypt interceptors so that when a user registers on the frontend, the backend scrambles the password string before saving it to Users.
2. **JWT Handshake:** Configured the login route to generate a cryptographic Token upon successful password verification, sending it back to the browser to be stored in the device's LocalStorage.
3. **Route Guards:** Wrapped the Dashboard and Admin React routes in a <ProtectedRoute> component that checks for a valid JWT before rendering.

Phase 4: The Admin Portal & Tracking Telemetry

1. **Frontend Hook:** Created a useAnalytics React hook that fires every time the user's Location (URL) change's, sending a ping to the backend's /api/track/page endpoint.
2. **Admin UI:** Built a dedicated, restricted layout for bunnyseshireddy@gmail.com using complex grid systems to display data tables.
3. **Action Handlers:** Wired buttons in the Admin UI (e.g., "Ban User") to corresponding DELETE or UPDATE API routes on the backend.

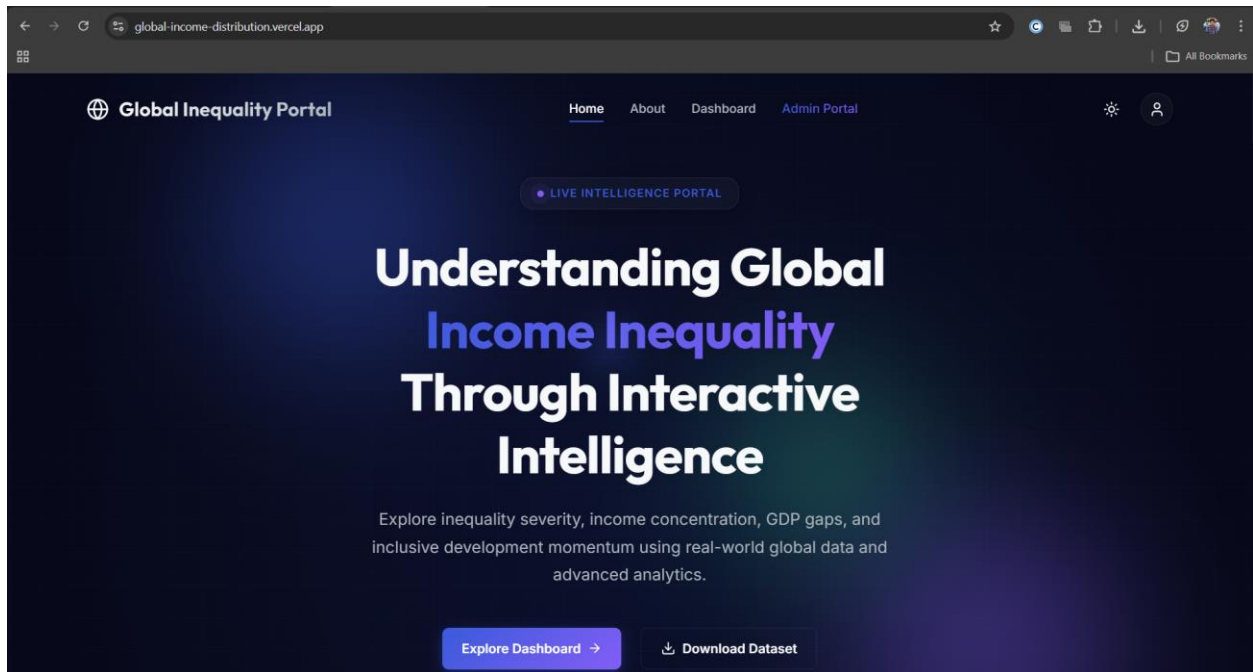
Phase 5: Final Polish & Production Deployment

1. **UX Refinement:** Replaced static social buttons with interactive "Coming Soon" toast notifications, added smooth momentum scrolling to modals, and ensured mobile responsiveness via CSS Media Queries.
2. **Environment Variables:** Abstracted hardcoded localhost URLs into .env configuration files so the app dynamically points to the live server in production.

3. **SPA Redirects:** Created vercel.json and Netlify _redirects files to fix React Router 404 errors during deployment.
4. **Push to Production:** Connected the GitHub repository to Vercel (Frontend) and Render (Backend) to execute automated builds and release the project to the public internet.

5. Conclusion

The Global Income Inequality Intelligence Portal successfully merges complex socioeconomic data with an accessible, high-performance web interface. By leveraging a modern React frontend and a lightweight, secure Node.js/SQLite backend, the platform delivers an engaging and educational experience. It not only visualizes critical global trends but also provides researchers and analysts with the robust administrative tools needed to securely manage users. The result is a premium, interactive dashboard that fulfills all project requirements while remaining easily scalable for future feature phases.



---- With this Milestone 3 is Completed Successfully ----